

UNIVERSIDAD NACIONAL DEL NORDESTE

Facultad de Ciencias Exactas y Naturales y Agrimensura

Informe Técnico Analítico

Sistema de Semáforos Inteligentes

Asignatura: **Paradigmas y Lenguajes de Programación**

Carrera: **Licenciatura en Sistemas de Información**

Profesores:

- Msc. Ricardo Monzón
- Ringa, Mónica - Rodríguez, Leandro Mascazzini
- Matías - Ramírez, Walter

Integrantes Grupo 43:

- Emmanuel David Otero
- Federico Gonzales
- Franco Ezequiel Veloso

Año Académico: 2026

Fundamentos del diseño funcional adoptado

El paradigma funcional impone un marco de pensamiento declarativo que modela el cómputo como la evaluación de funciones matemáticas, evitando de manera explícita el cambio de estado y la mutabilidad de los datos. Para la construcción de este sistema, se aplican de manera estricta los siguientes puntos:

Inmutabilidad y Transparencia Referencial: Ninguna de las funciones desarrolladas altera variables globales ni estructuras de datos preexistentes en memoria (ausencia total de efectos colaterales). Toda la información es tratada como datos de lectura y las transiciones producen nuevos valores sin destruir los anteriores.

Determinismo Algorítmico: Una función se considera matemáticamente "pura" si, dados los mismos argumentos de entrada, retorna exactamente idéntico resultado sin importar el contexto temporal o el estado de la máquina.

Composición y Modularidad Local: Se prioriza el uso de enlaces locales mediante la primitiva LET, aislando los cálculos internos sin contaminar el entorno de ejecución global.

Justificación de librería seleccionada: local-time

Para esta segunda etapa del trabajo práctico, como grupo decidimos integrar la librería externa local-time a través del gestor de paquetes Quicklisp.

El principal motivo por el que elegimos esta herramienta es porque nos resolvió el problema de fechas para el requerimiento de la auditoría. Al principio vimos que Common Lisp trae funciones nativas para el tiempo (como get-universal-time), pero son bastante limitadas: calculan los segundos desde el año 1900 y es muy tedioso tener que convertir a mano esos números para que se vean bien en pantalla.

Usar local-time nos dio dos ventajas enormes:

1. **Maneja el estándar Unix:** Trabaja con los segundos transcurridos desde 1970. Como nuestro sistema de semáforos ya venía calculando los cambios en base al tiempo-unix, la librería encajó perfecto con nuestra lógica matemática sin tener que hacer conversiones extrañas.
2. **El formato directo:** Gracias a su función format-timestring, pudimos imprimir en consola exactamente lo que pedía la consigna (el formato [YYYY-MM-DD HH:MM:SS]) de manera directa. Si no usábamos esta librería, íbamos a tener que armar funciones larguísimas y llenas de concatenaciones solo para acomodar los guiones y los dos puntos del reloj.

En conclusión, la elección de local-time nos permitió mantener el código mucho más limpio, corto y entendible, respetando la idea de modularidad que nos pide la cátedra en el paradigma funcional.

BITÁCORA DE DEPURACIÓN

A lo largo del desarrollo de las Fases y Requerimientos del Sistema, el equipo documentó y resolvió los siguientes incidentes técnicos:

Bug #1: Error de Directorio Raíz en Git

- **Problema:** Al intentar sincronizar el repositorio remoto mediante el comando git pull o verificar el estado con git status, la terminal devolvió el error fatal: **not a git repository (or any of the parent directories): .git**.
- **Causa:** Tras realizar el git clone, el directorio de trabajo activo de la terminal permaneció en la carpeta padre, la cual no estaba inicializada como repositorio Git.
- **Resolución:** Se estableció como norma del equipo abrir el entorno de desarrollo apuntando directamente a la raíz de la carpeta clonada (ej. TP-INTEGRADOR). Al abrir la terminal integrada desde allí, el motor de Git reconoció exitosamente la estructura del proyecto.

Bug #2: Archivo de Entorno LISP No Encontrado

- **Problema:** Al intentar instalar el gestor de paquetes mediante (load "quicklisp.lisp"), el motor SBCL arrojó una interrupción de tipo **SB-INT:SIMPLE-FILE-ERROR: file does not exist**.
- **Causa:** El intérprete LISP se estaba ejecutando en una ruta del sistema diferente a la ubicación real donde se había descargado el archivo script de instalación.
- **Resolución:** Se procedió a cerrar la instancia actual de SBCL con (quit). Luego, dentro del explorador de archivos del editor, se hizo clic derecho sobre el archivo .lisp específico seleccionando *"Abrir en terminal integrada"*. Esto garantizó la alineación absoluta de las rutas antes de volver a invocar sbcl y cargar el archivo.

Bug #3: Error de Paquete Quicklisp

- **Problema:** Al ejecutar la primera instrucción de la Fase 2 (ql:quickload "local-time"), el compilador detuvo la ejecución arrojando el error **Package QL does not exist**.
- **Causa:** El gestor de librerías Quicklisp no se encontraba cargado en la memoria del motor de LISP, ya que no se había configurado su arranque automático.
- **Resolución:** Se cargó manualmente el script de configuración interno ejecutando (load "~/quicklisp/setup.lisp") en el REPL. Inmediatamente después, se ejecutó el comando (ql:add-to-init-file) para forzar a SBCL a incluir Quicklisp en su archivo de inicio, resolviendo el problema de forma permanente para futuras sesiones.

Bug #4: Colisión con Símbolos Reservados del Sistema

- **Problema:** Durante la compilación de la lógica temporal, el *debugger* se activó mostrando un **SYMBOL-PACKAGE-LOCKED-ERROR: Lock on package SB-EXT violated**.
- **Causa:** Se definió una función propia utilizando el identificador timer (defun timer ...). LISP posee una función interna fuertemente protegida con ese mismo nombre para la medición de sus procesos del sistema, bloqueando la sobreescritura por motivos de seguridad.
- **Resolución:** Aplicando buenas prácticas de nombrado de variables, se modificó el identificador de la función a un término en español, pasando de timer a temporizador. Tras esto, la recarga del archivo se ejecutó sin colisiones.

Bug #5: Retorno NIL en Selección Condicional

- **Problema:** Al realizar la prueba de escritorio de la función (auditoría 30), en lugar de devolver la cadena de texto con el registro del cambio de luz, la función devolvía un **NIL** vacío.
- **Causa):** Inconsistencia en la comparación de los símbolos. La función pura retornaba estados con prefijos (ej. 'en-rojo), pero la estructura COND dentro de la función de auditoría evaluaba igualdades simples sin el prefijo ((eq estado-actual 'rojo)). Al no cumplirse ningún caso, COND retornaba por defecto **NIL**.
- **Resolución:** Se reescribieron los parámetros de la estructura COND unificando la nomenclatura de símbolos del programa. Se añadieron los prefijos correspondientes ('en-rojo, 'en-verde, 'en-amarillo) para que la evaluación booleana a través de eq fuera matemáticamente verdadera.

Bug #6: Choque de Historiales (Merge Conflict) al hacer Push

- **Problema:** Al intentar subir avances a GitHub, se rechazó el push y, tras intentar un git pull, surgió el error **Merge conflict in lisp/core.lsp**.
- **Causa:** Dos integrantes del equipo editaron líneas del mismo archivo (core.lsp) en simultáneo, provocando que Git pausara la fusión automática para evitar la pérdida de código funcional.
- **Resolución:** Se encapsularon los cambios locales en un commit seguro. Luego, mediante la interfaz visual de VS Code, se analizaron las líneas conflictivas demarcadas por <<<<<< HEAD. Se unificó el código validando la lógica de ambas partes (eligiendo "Accept Both Changes"), se guardó el archivo y se finalizó el proceso consolidando el código con un *Merge Commit* limpio antes de realizar el push definitivo a la rama main.

Introducción a Haskell y su impacto

Un lenguaje de programación funcional puro que nació a finales de los 80 pero que sigue siendo un referente clave cuando buscamos robustez. Lo que me parece más interesante de su filosofía es cómo combina tres pilares: el tipado estático fuerte, la evaluación perezosa (lazy evaluation) y la inmutabilidad de los datos.

A diferencia de lo que ocurre con lenguajes como Common Lisp, donde el tipado es más flexible, Haskell nos obliga a declarar y respetar los tipos de datos desde el primer momento. Esto, en el día a día del desarrollo, es una ventaja enorme: nos permite detectar y corregir una gran cantidad de errores directamente durante la fase de compilación, mucho antes de que el código llegue a ejecutarse.

- Aplicaciones en el mundo real : Aunque a veces se percibe como un lenguaje

exclusivamente académico, he podido verificar que su uso está muy consolidado en sectores donde la confiabilidad y la corrección matemática son críticas. No es un lenguaje de nicho sin aplicaciones prácticas; de hecho, grandes organizaciones lo implementan en entornos de producción exigentes:

- **Sistemas Financieros:** Empresas como Standard Chartered, Mercury y Tsuru Capital lo utilizan para suavizar riesgos en operaciones complejas.
- **Infraestructura Tecnológica:** Meta lo aprovecha para el desarrollo de herramientas internas y análisis de datos a gran escala.
- **Blockchain:** IOHK / Input Output Global lo eligió como el núcleo para desarrollar la red Cardano, precisamente por la seguridad que ofrece.
- **Conclusión y valoración**

Desde nuestro punto de vista, las principales ventajas que aporta Haskell se resumen en tres puntos: una seguridad de tipos envidiable, la facilidad para razonar matemáticamente sobre el comportamiento del software y, en consecuencia, una reducción de los errores en el tiempo de ejecución. Es una herramienta diseñada para cuando el margen de error debe ser igual a cero.

Preguntas Teóricas

1. ¿Cómo modelaron los colores del semáforo y qué es un ADT?

En nuestro código de Lisp, manejamos los colores dinámicamente usando símbolos como 'en-rojo o 'en-verde. Lisp no revisa esto hasta que ejecutas el programa. En Haskell, la forma correcta de hacerlo es usando un Algebraic Data Type (ADT). Se declara así: `data ColorSemaforo = Rojo | Amarillo | Verde` Un ADT nos permite definir explícitamente cuáles son los únicos valores válidos. La ventaja es que si en nuestra función de transición nos olvidamos de codear qué pasa con el Amarillo, o le pasamos por error un estado Azul, el compilador de Haskell directamente te frena y no te deja compilar.

2. La Evaluación Perezosa con un flujo infinito de tráfico:

La evaluación perezosa (Lazy Evaluation) significa que Haskell solo procesa las cosas en el momento exacto en que necesita mostrar un resultado. Si nosotros enchufáramos nuestro semáforo a un flujo "infinito" de autos y eventos de tiempo, en un lenguaje de evaluación estricta la memoria explotaría al intentar cargar todo. En Haskell no pasa eso: el programa iría "consumiendo" y procesando los eventos del temporizador segundo a segundo de manera eficiente, lo que es ideal para un sistema que tiene que correr para siempre en la calle.

Conclusión

La experiencia de comparar Lisp con Haskell fue interesante para ver los dos extremos del paradigma funcional. Lisp es más dinámico, nos permitió escribir y probar rápido las funciones con símbolos y listas, pero también daba lugar a equivocarnos por pasar mal

un dato. Haskell nos pareció mucho más rígido y tiene una curva de aprendizaje inicial más difícil porque te obliga a diseñar todos los tipos de datos antes de programar la lógica. De todas formas, entendemos por qué se usa: la tranquilidad de que si el compilador de Haskell te aprueba el código, es casi seguro que no se va a romper en tiempo de ejecución,

Bibliografía:

Monzón, R. (2026). *Apuntes de Cátedra: Paradigmas y Lenguajes de Programación*. Universidad Nacional del Nordeste (UNNE)

Sussman, G. J., & Abelson, H. (1996). *Estructura y interpretación de programas de computadora*. McGraw-Hill.

Chacon, S., & Straub, B. (2014). *Pro Git: Control de versiones con Git* (2da ed.). Apress.

Quicklisp. (2026). *Quicklisp documentation: Managing Common Lisp libraries*. Recuperado de <https://www.quicklisp.org/beta/>